OOPS

Class: 16
07/05/2025

(Object Oriented Programming)

→ Programme in java completely based on class and object.

→ Is java depends on OOPS – No, not 100%. Object Oriented programming, due to,

↓

Primitive datatypes are not classes in Java.

↓

to change that datatypes into classes they brought

Wrapper classes.

→ **Class**: It is a blueprint of a file (i.e, programme).

→ Combination of 3 parts: (without these three, we can create class)

1. Variables (Fields)
2. Methods (Functions).
3. Constructor.

Ex: Car, Vachicle, Human, Office, School etc.

[Common things are written in a class].

Object: Implimentation of blueprint. [All objects stored in Heap Memory].

* One class can have infinite object.

*Ex:

```
package com;
public class Bike {
    String colour;
    int gear;
    int speed;
    double petrolcapacity;

    Void increaseGear( ){
        gear++;
```

Fields(8) Variables

(Functions)

[If the methods are in same package, we need not to import methods]

[If they are in different package, we have to import].

}

```
void accelerate ( ) {
    speed += 20;

}
void applyBrake( ) {
    gear--;
    speed -= 10;

}
```

[we can't write break directly, due to that it is a keyword in Java]

[keywords of Java cannot write as method name].

```
package com;
public class Test {
    public static void main (String [ ] args) {
        Bike myBike = new Bike ( );
        myBike.colour = "Black";
        System.out.println(myBike.colour);
```

[new is a keyword, used to create an object]

↳ constructor is java (main reason to create an object in java).

output: Black

```
        myBike.gear = 0;
        myBike.speed = 0;
        myBike.petrolCapacity = 12;
        Syso (myBike.gear);
        Syso (myBike.speed);
        Syso (myBike.petrolCapacity);
```

output: Black
0
0
12.0

Methods Calling objects:

```
myBike.increaseGear( );
    Syso( myBike.gear); → output: 1
myBike.accelerate( );
    Syso (myBike.speed); → output: 20

myBike.applyBrake( );
    Syso(myBike.gear); → output: 0
    Syso(myBik.speed); 10
```

```
Bike friendBike = new Bike( );
friendBike.gear = 1;
friendBike.colour = "Red";
friendBike.speed = 20;
friendBike.petrolCapacity = 20;
    Syso(friendBike.colour); output: Red.
```

method calling:

```
friendBike.accelerate( );
    Syso(friendBike.speed); → output: 40.
```

* When we create a class, memory will not assign to taken variables.
* Memory will assign only when we create an object.
* A class cannot (taken) consume any memory until and unless we create an object.

Constructor:
* Responsible to create an object.
* Responsible to assign values to an instance variables.
* It looks like a method.

Syntax:

```
public class Bike {                    → class name.
    Bike( ) {                          → Non-parameterized constructor.
                                         (or) default constructor.
    }                                  → constructor.
    Bike(String colour) {              → parameterized constructor.
    }
}
```

* Constructor will not have a Return type (like void etc).
* Constructor should always have, class name, must be same name as classname & constructor ~~same~~ same.
* JVM provides default constructor, we can create an object without giving constructor.
* A class can have multiple constructors, but we cannot create same constructor, we can pass multiple parameters (should not be same).

ex:
```
public class Bike {
    Bike() {
    }
    Bike(String colour) {
    }
    Bike(int speed; String colour) {
    }
}
```

* when we write a parameterized constructor, JVM will not give default constructor.

Ex:
```
public class Bike {
    String colour;
    int gear;
    int speed;
    double petrolCapacity;

    ~~Bike(String co~~
    Bike() {
        syso("Creating an object....");
    }
    Bike(String new colour) {
        colour = new colour;
    }
```

[; is used in java to terminate a line.]

```
PSVM {
    Bike myBike = new myBike();
    Bike secondBike = new Bike("Blue");
    Syso(secondBike.colour);
```

output: Bike. creating an object....

* We can ~~cons~~ create only one non-parameterised constructor.